

API Reference

Every reflected class, function and property, generated from the shipping headers.

NOTE

Generated by `ULcmNavDocCoverageCommandlet ( -run=LcmNavDocCoverage - DumpMarkdown=... )`. Source of truth is the C++ headers in `Plugins/LCMNav3D/Source/LCMAINavigation/Classes/` (plus `Public/`); this file is a reflection-derived view and can be regenerated at any time.

Coverage summary

Symbol kind	Total	With tooltip	Missing
UCLASS	109	109	0
UFUNCTION	65	65	0
UPROPERTY	343	343	0
Total	517	517	0

Coverage: 100.00%

Reflected symbols by class

LcmAIPerceptionBridge

Mirrors UAIPerceptionComponent-sensed actors into a decaying threat tracker for the tactical EQS surface.

FUNCTIONS

- `HandleTargetPerceptionUpdated`: Bound to the paired UAIPerceptionComponent's OnTargetPerceptionUpdated.
- `GetTrackedThreats`: Current tracked-threat actors (post-decay).

PROPERTIES

- `DecayWindowSeconds`: Seconds a lost-sight threat stays queryable before it decays out.
- `PerceptionComp`: The paired perception component (found on the owner at BeginPlay).

LcmBTDecorator_ChunkLoaded

Decorator that passes only when the SVO chunk containing the queried world position is currently loaded (cheap single-endpoint hash lookup, no path probe).

PROPERTIES

- **QueryLocationKey** : Blackboard key holding the world position to probe. Accepts Vector or Object (Actor location is used in the latter case).
- **bRequireFullResolution** : When true (default), only fully-resolved chunks (post- promotion) satisfy. When false, any loaded chunk does.

LcmBTDecorator_HasLineOfSight

Decorator that passes when the agent has volumetric line of sight to the target via the Lazy Theta* LOS walker, honouring AgentRadius through the JFA clearance gate.

PROPERTIES

- **TargetKey** : Blackboard key for the LOS target. Vector = world location, Object = AActor (its current location is used).
- **AgentRadius** : Agent collision radius for the JFA clearance gate. Higher = more conservative (rejects narrow gaps).

LcmBTDecorator_IsInCover

Decorator that passes when the agent is in cover relative to a threat: its leaf SweepPenalty is at or below MaxSweepPenalty AND the threat's line of sight to it is blocked.

PROPERTIES

- **ThreatKey** : Threat key. Vector = world location, Object = AActor.
- **MaxSweepPenalty** : Highest SweepPenalty (0..255) the agent's leaf may carry to count as safe. Default 32 ≈ "lightly swept, mostly safe".
- **AgentRadius** : Agent collision radius for the LOS walker's clearance gate.

LcmBTDecorator_PathLengthBelow

Decorator that passes when the true-3D LCM Nav3D path distance from the agent to the goal is below MaxLengthCm (uses the synchronous tactical-library path probe; do not run per-frame).

PROPERTIES

- **GoalKey** : Blackboard key for the goal endpoint. Vector = world location, Object = AActor (its current location is used).
- **MaxLengthCm** : Maximum allowed path length, in centimetres, for the decorator to pass.
- **AgentRadius** : Agent collision radius (cm) used by the path probe's clearance gate.

LcmBTDecorator_ThreatVisibleViaPerception

Decorator that passes when AIPerception currently perceives a hostile AND the agent has clear volumetric LOS to it (the "can-shoot-now" gate).

PROPERTIES

- **ThreatKey** : Optional explicit threat key. When unset (NAME_None), the decorator uses AIPerception's closest currently-perceived actor.
- **AgentRadius** : Agent collision radius (cm) used by the LOS walker's clearance gate.

LcmBTService_NavigationLoad

Service that each tick writes the navigation manager's SVO streaming stats into a blackboard key so designers can gate expensive LCM Nav3D queries on idle streaming.

PROPERTIES

- **LoadedChunkCountKey** : Int blackboard key receiving the count of currently loaded SVO chunks.

LcmBTService_PathProgressMonitor

Service that each tick writes a Euclidean approach-progress fraction ($1 - \text{dist}/\text{initial_dist}$) and remaining distance to the goal into blackboard keys.

PROPERTIES

- **GoalKey** : Blackboard key for the goal. Vector = world location, Object = AActor (its current location is used).
- **ProgressKey** : Float blackboard key receiving the progress fraction (0..1) toward the goal.
- **RemainingCmKey** : Float blackboard key receiving the remaining straight-line distance (cm) to the goal.
- **GoalChangeTolerance** : Distance (cm) the goal must move before the progress baseline is re-anchored.

LcmBTService_SquadSnapshot

Service that each tick aggregates perceived allies within SquadRadius and writes ally count, squad centroid, and cohesion radius into blackboard keys.

PROPERTIES

- **AllyTag** : Tag filter. Only perceived actors with this tag count as ally.
- **SquadRadius** : Radius (cm) around the agent within which allies are counted.
- **bIncludeSelf** : When true, the agent itself is included in the squad aggregate.
- **NumAlliesKey** : Int blackboard key receiving the number of allies in the squad.
- **CentroidKey** : Vector blackboard key receiving the squad's centroid (average ally position).
- **CohesionRadiusKey** : Float blackboard key receiving the cohesion radius (max ally distance from the centroid, cm).

LcmBTService_ThreatFieldSampler

Service that each tick samples the SVO SweepPenalty risk field at the agent's location, writing the threat level and an "in threat zone" flag ($\text{level} \geq \text{AlertThreshold}$) into blackboard keys.

PROPERTIES

- **ThreatLevelKey** : Int blackboard key receiving the current SweepPenalty (0..255).
- **InThreatZoneKey** : Bool blackboard key receiving $\text{Level} \geq \text{AlertThreshold}$.
- **AlertThreshold** : SweepPenalty value (0..255) at or above which the agent counts as in a threat zone.

LcmBTTask_ChunkPrewarm

Task that proactively forces SVO chunk generation at a target location and waits (up to TimeoutSeconds) for the chunk to become ready before succeeding.

PROPERTIES

- **TargetKey** : Blackboard key for the location whose chunk to prewarm. Vector = world location, Object = AActor (its current location is used).
- **bRequireFullResolution** : When true, the task waits for the chunk to be fully resolved (post-promotion); when false, any loaded chunk satisfies.
- **TimeoutSeconds** : Maximum time (seconds) to wait for the chunk before the task fails.

LcmBTTask_DynamicObstaclePush

Task that steers the agent down the SweepPenalty gradient (away from rising threat) until it clears the obstacle or times out.

PROPERTIES

- **PushSpeed** : Movement speed (cm/s) while pushing away from the obstacle.
- **ProbeStep** : Step distance (cm) used to sample the SweepPenalty gradient around the agent.
- **ExitThreshold** : SweepPenalty value (0..255) at or below which the agent is considered clear and the task succeeds.
- **MaxDuration** : Maximum time (seconds) to push before the task succeeds and yields.

LcmBTTask_EvadeCBS

Task that retreats from the threat gradient using context-based steering (CBS) over SVO danger sampling, blending an anti-threat direction with ray-evaluated safe headings until clear or timed out.

PROPERTIES

- **EvadeSpeed** : Movement speed (cm/s) while evading.
- **ProbeStep** : Step distance (cm) used to sample the SweepPenalty gradient around the agent.
- **ExitThreshold** : SweepPenalty value (0..255) at or below which the agent is considered safe and the task succeeds.
- **MaxDuration** : Maximum time (seconds) to evade before the task succeeds and yields.
- **AgentRadius** : Agent collision radius (cm) used when evaluating SVO danger for the CBS rays.
- **LookAheadDistance** : Distance (cm) ahead along each CBS ray over which SVO danger is sampled.
- **NumCBSRays** : Number of context-based-steering rays cast around the agent to select a safe heading.

LcmBTTask_FindCover

Behavior Tree task that locates a navigable SVO leaf with line-of-sight blocked from the blackboard enemy actor (i.e. cover). Returns Succeeded once a valid cover location is written back to the blackboard, or Failed if no cover within **SearchRadius** qualifies.

PROPERTIES

- **EnemyKey** : The Blackboard Key for the Enemy actor (Who are we hiding from?)
- **SearchRadius** : Search Settings (Exposed to Behavior Tree)
- **CoverHeightOffset** : Vertical offset (cm) added to candidate cover positions so the pawn stands above the cover floor rather than embedded in it.
- **bPrioritizeClosest** : When true, the closest qualifying cover position wins. When false, the search picks the first qualifying candidate (lower latency).

LcmBTTask_FlyTo

Behavior Tree task that flies an AI pawn from its current position to a blackboard target via the LCMNav3D pathfinder. Owns one async path request token (re-issued on dynamic replanning), runs per-tick steering (configurable flight style + avoidance style + wall avoidance + swarm separation), and finishes the task on arrival within **AcceptanceRadius**.

FUNCTIONS

- **OnPathFound** : The Callback function for Async Pathfinding

PROPERTIES

- **AcceptanceRadius** : Settings exposed to the Behavior Tree Node
- **FlightSpeed** : Cruise speed of the pawn along the path, in cm/s.
- **QueryOptions** : Per-request pathfinding options (solver choice, GPU on/off, smoothing, heuristic mode). Forwarded as-is into **FindPathAsync**.
- **SeparationRadius** : How close is "too close" to another agent? (e.g. 150 units)
- **SeparationWeight** : How hard should we push away from neighbors? (0.0 = No push, 2.0 = Strong push)
- **bUseFlightMovementComponent** : [P5] Delegate locomotion to a ULcmFlightMovementComponent on the pawn (for ultra-realistic 6-DOF drone/bird/fish dynamics). When ON and the pawn has an enabled component, this task only produces the guidance velocity (pure-pursuit + avoidance + arrival slowdown) and the component owns position + orientation. OFF (default) = the built-in kinematic integrator below. The kinematic-only knobs below (accel/banking/flight style) hide when this is ON: the component owns them via its FlightConfig.
- **MaxAcceleration** : How fast can we accelerate? (Higher = Snappier, Lower = Drifty/Heavy) [Kinematic only; owned by the flight component when delegating.]
- **BankingAmount** : How much do we bank into turns? (Visual only) [Kinematic only.]
- **MaxBankingAngle** : Max banking angle in degrees (e.g., 45 degrees) [Kinematic only.]
- **DecelerationDistance** : Distance to start slowing down (Arrival behavior). Still used when delegating: it shapes the guidance target speed near the goal.

- **FlightStyle** : Choose how this agent moves [Kinematic only; the component's FlightConfig has its own model/style when delegating.]
- **AvoidanceTimeHorizon** : How many seconds into the future do we look? (Standard: 2.0s)
- **CollisionMargin** : Radius multiplier for safety (e.g. 1.2 = Avoid by 20% extra margin)
- **AvoidanceStyle** : Choose how this agent avoids neighbors
- **WallCheckDistance** : WALL AVOIDANCE How far to check for walls? (e.g. 150 units)
- **WallPushForce** : How hard to push away from walls? (Must be strong!)
- **PathCenteringForce** : PATH CORRECTION How hard to pull agents back to the center line? (e.g. 500) 0 = Free drift (current behavior) 1000 = Strict line following (Train on tracks)
- **bEnableGoalPrediction** : When true, AND `r.LcmNav.Chase.Enable != 0`, the task resolves moving Object-key goals through `FLcmGoalProvider` and uses constant-velocity lead prediction (linear-extrapolation intercept) as the planner goal. When false, the legacy snapshot behavior is preserved bit-identically.
- **LeadSecondsCap** : Upper bound on lead time (seconds). Forwarded as `LeadSecondsCap` to `FLcmGoalProvider::GetLeadPosition`. Clamps blow-up when the target is receding faster than the agent can close. Override at runtime via `r.LcmNav.Chase.LeadSecondsCap (>= 0)`. Typical 0.5..2.0.
- **DriftReplanCm** : Distance (cm) between the last-planned goal and the current lead position that triggers an automatic replan. Override at runtime via `r.LcmNav.Chase.DriftReplanCm (>= 0)`.
- **bForceEnableChase** : [Fab marketplace UX, 2026-05-29] Per-task override for the `r.LcmNav.Chase.Enable` CVar gate. When true, chase activates from `bEnableGoalPrediction` alone, so buyers don't need to know the CVar exists. Default false preserves the legacy two-gate behavior (CVar AND `bEnableGoalPrediction`) bit-identically. The CVar is retained as a project-wide kill switch / debug toggle.

LcmBTTask_FollowFlowField

Task that computes an LCM Nav3D path to the goal and follows its waypoints (LinearDirect steering) until arrival or timeout.

PROPERTIES

- **GoalKey** : Blackboard key for the goal. Vector = world location, Object = AActor (its current location is used).
- **MoveSpeed** : Movement speed (cm/s) along the followed path.
- **WaypointAdvanceRadius** : Distance (cm) within which the current waypoint is considered reached and the next is selected.
- **ArrivalRadius** : Distance (cm) from the goal at which the task succeeds.
- **AgentRadius** : Agent collision radius (cm) used by the path probe's clearance gate.
- **MaxDuration** : Maximum time (seconds) to follow the path before the task fails.

LcmBTTask_Land

Task that descends the agent vertically at `DescentRate` until it reaches the `TouchDownZ` world height, then succeeds.

PROPERTIES

- **TouchDownZ** : Target world Z height (cm) to descend to.
- **DescentRate** : Vertical descent speed in cm/s.
- **TouchDownTolerance** : Distance (cm) above `TouchDownZ` at which the agent counts as landed.

LcmBTTask_OrbitTarget

Task that circles the agent around a target, holding `OrbitRadius` while moving tangentially at `OrbitSpeed`.

PROPERTIES

- **TargetKey** : Blackboard key for the orbit centre. Vector = world location, Object = AActor (its current location is used).
- **OrbitRadius** : Distance (cm) the agent maintains from the target while orbiting.
- **OrbitSpeed** : Tangential movement speed (cm/s) around the orbit.
- **bClockwise** : When true, orbit clockwise (viewed from above); when false, counter-clockwise.

- `bMatchTargetZ` : When true, the agent matches the target's world Z height while orbiting; when false, it holds its own height.

LcmBTTask_Patrol

Task that moves the agent through a list of waypoints in order, pausing at each, optionally looping.

PROPERTIES

- `Waypoints` : Ordered list of patrol waypoints, in world coordinates.
- `MoveSpeed` : Movement speed (cm/s) between waypoints.
- `ArrivalRadius` : Distance (cm) within which a waypoint counts as reached.
- `DwellSeconds` : Time (seconds) the agent dwells at each waypoint before advancing.
- `bCycle` : When true, the patrol loops back to the first waypoint after the last; when false, it ends.

LcmCostParityCommandlet

Cost-quantization parity commandlet: Gate 1 BLOCKER. Generates 1000 deterministic (start, end, neighbor) triples and asserts that CPU `QuantizeEdgeCost` and the equivalent GPU compute-shader path produce bit-identical uint32 quantized values across all three heuristic modes (Euclidean, Manhattan, Chebyshev). Any tie-break mismatch surfaces here. See header for invocation, exit codes, and the broader gate context.

LcmCrowdAvoidanceBridge

Component that registers its owning LCM Nav3D pawn with UCrowdManager as an ICrowdAgentInterface provider so Detour RVO crowd agents avoid it, with a Z-band group filter that drops false cross-altitude collisions.

FUNCTIONS

- `UnregisterFromCrowd` : Manually remove this agent from UCrowdManager so it stops being an avoidance neighbour (e.g. gate participation by gameplay state).
- `RegisterWithCrowd` : Manual register / unregister for projects that want to gate participation by gameplay state (e.g. only enable while alive).

PROPERTIES

- `bAutoRegister` : Auto-register with UCrowdManager on BeginPlay. Default true.
- `AgentRadius` : Agent cylinder radius for Detour RVO (cm).
- `AgentHalfHeight` : Agent cylinder half-height for Detour RVO (cm). Keep small so LCM Nav3D flying agents don't tower-overlap with ground agents during the per-tick RVO query (Z-band filter is the primary mechanism; this is a secondary safeguard).
- `MaxSpeed` : Max speed reported to Detour (cm/s).

LcmDebugCoverViz

Debug component that periodically runs FindBestCover and draws the chosen cover spot plus agent/threat connecting lines; gated by `r.LcmNav.Debug.CoverViz`.

PROPERTIES

- `OverrideThreatActor` : When non-null, overrides the perception-derived threat actor.
- `SearchRadiusCm` : Search radius around the agent for the FindBestCover query (cm).
- `CoverHeightOffset` : Cover-finder height-offset parameter passed through to FindBestCover.
- `RefreshHz` : Refresh cadence. Higher = smoother viz, more cover queries.

LcmDebugFlowFieldViz

Debug component that draws a grid of arrows showing the flow-field gradient toward GoalLocation around the agent; gated by `r.LcmNav.Debug.FlowFieldViz`.

PROPERTIES

- `GoalLocation` : World position toward which the visualised gradient flows.
- `GridHalfExtentCm` : Half-extent of the sample grid drawn around the agent (cm).
- `GridStepCm` : Step between sample points along each axis (cm).
- `ArrowLengthCm` : Length (cm) of each direction arrow.
- `RefreshHz` : Refresh cadence.

LcmDebugStateTreeOverlay

Debug component that draws a per-agent navigation-state summary (velocity, chunk, perceived-target count, path token) above the pawn; gated by `r.LcmNav.Debug.StateTreeOverlay`.

PROPERTIES

- `OverlayHeightOffset` : Vertical offset (cm) above the pawn's root where the overlay is drawn.
- `RefreshHz` : Refresh cadence (Hz). Higher = smoother text update, more CPU.

LcmDeterminismHelpers

Process-wide deterministic-random helpers (seeded stream + tie-break) gated on `r.LcmNav.Determinism.Seed`, for replay-safe and networked-deterministic navigation randomness.

LcmDynamicObstacleComponent

Marks an actor as a dynamic LCMNav3D obstacle.

Two operating modes, selected by `r.LcmNav.DynObs.EventDriven`:

1. EVENT-DRIVEN (default, `EventDriven=1`) binds to `RootComponent->TransformUpdated`. Per-event the component unions the new bounding box into a pending dirty region and arms a one-shot coalesce timer (`r.LcmNav.DynObs.CoalesceMs`, default 33 ms = one frame at 30 fps). On timer expiry the union of old-footprint + all coalesced new-footprints is flushed in a single `UpdateDynamicObstacle` call. Component tick is disabled in this mode. Zero CPU cost when the actor is stationary; instant response (within one coalesce window) when it moves.
2. POLLING (`EventDriven=0`) is the legacy behavior: `TickComponent` at 10 Hz polls `GetActorLocation` and fires only when displacement exceeds `UpdateDistanceThreshold` (default 50 cm). Provided for ablation / deterministic-test scenarios where event jitter would perturb traces.

Both modes end in the same `UpdateDynamicObstacle` call; only the trigger differs.

FUNCTIONS

- `RemoveTrackedComponent` : Runtime/BP removal of a previously added sub-component. Components listed in `TrackedComponents` (the designer list) are not affected.
- `AddTrackedComponent` : Runtime/BP registration of an obstacle sub-component (e.g. a child mesh attached after `BeginPlay`). Idempotent; rebinds immediately.

PROPERTIES

- `UpdateDistanceThreshold` : [Polling mode only] How far does the object need to move before triggering an SVO rebuild? Keep around half your `MinVoxelSize` (~50 cm). Ignored when `r.LcmNav.DynObs.EventDriven=1` (event-driven mode uses a coalesce timer instead of a distance threshold).
- `TrackedComponents` : Designer-facing list. Pick the collision/mesh components that actually represent this obstacle. Empty = whole actor (legacy).

LcmDynamicObstacleSubsystem

P8b: Zero-config auto-attach of `ULcmDynamicObstacleComponent`.

A `UWorldSubsystem` that:

1. On `Initialize`, iterates every existing actor in the world and (per the filter predicate `ShouldAutoAttach`) attaches a `ULcmDynamicObstacleComponent` to those that qualify.

2. Binds `UWorld::AddOnActorSpawnedHandler` so future-spawned actors are considered too. This handles streamed-in levels, blueprint-spawned props, physics-spawned debris, etc.

Gated by `r.LcmNav.DynObs.AutoAttach` (default 1). Disabling reverts to the pre-manual-attach model bit-identically: no scans, no handlers, no attached components for newly spawned actors. (Already-attached components are NOT removed; they keep working.)

Filter predicate semantics (see `ShouldAutoAttach` docstring for the full table): Movable mobility + collision enabled + responds Block to either `WorldStatic` or `WorldDynamic`. Rejects the nav manager itself, brushes, volumes, and any actor already carrying a `ULcmDynamicObstacleComponent`.

EnvQueryContext_LcmKnownThreats

EQS context exposing tag-marked threat actors as query targets for tactical tests.

PROPERTIES

- `ThreatTag` : Actors carrying this tag are treated as threats.

EnvQueryContext_LcmThreatsFromAIPerception

EQS context exposing the querier's AI-perception-tracked threats as query targets.

EnvQueryGenerator_LcmHemisphericalRing3D

Hemispherical-ring shell of unblocked 3D points around a context.

PROPERTIES

- `GenerateAround` : Context the shell is centred on (the target; defaults to the querier).
- `RadiusMin` : Inner radius of the sampling shell (cm).
- `RadiusMax` : Outer radius of the sampling shell (cm).
- `NumRadialSteps` : Number of concentric radial steps between `RadiusMin` and `RadiusMax`.
- `NumInclinationRings` : Number of inclination (polar/elevation) rings from horizon to pole.
- `NumAzimuthSectors` : Number of azimuth (longitudinal) sectors around each ring.
- `bUpperHemisphereOnly` : Restrict to the upper hemisphere (high-ground / perch queries).

EnvQueryGenerator_LcmPointsAlongPath

Unblocked points sampled along the LCM Nav3D path from one context to another.

PROPERTIES

- `FromContext` : Path start (defaults to the querier).
- `ToContext` : Path goal (set this to your destination context).
- `AgentRadius` : Agent capsule radius used to plan the LCM Nav3D path that is sampled (cm).
- `Spacing` : Distance between consecutive sample points along the path (cm).

EnvQueryGenerator_LcmPortalProximity

Candidate points at macro-graph portal centres near a context (chokepoints).

PROPERTIES

- `GenerateAround` : Context the portal search is centred on (defaults to the querier).
- `Radius` : Only portals within this radius of the context are emitted (cm).

EnvQueryGenerator_LcmPointsInSVOVolume

Generates uniform-density candidate points in an AABB, filtered to unblocked SVO volume leaves.

PROPERTIES

- `GenerateAround` : Context the sampling volume is centred on (defaults to the querier).

- **HalfExtent** : Half-extent of the sampling AABB (cm).
- **Spacing** : Grid spacing between candidate points (cm).

EnvQueryTest_LcmLineOfSight3D

Visible to the context via true-3D SVO line of sight. Boolean.

PROPERTIES

- **Context** : Context to test visibility against (defaults to known threats).
- **AgentRadius** : Agent capsule radius used for the volumetric SVO LOS sweep (cm).

EnvQueryTest_LcmReachability3D

An LCM Nav3D path exists from the context to the item. Boolean.

PROPERTIES

- **Context** : Context the path is planned from (defaults to the querier).
- **AgentRadius** : Agent capsule radius used when testing for a valid LCM Nav3D path (cm).

EnvQueryTest_LcmPathDistance3D

LCM Nav3D path distance (NOT euclidean) from the context to the item. Float (cm).

PROPERTIES

- **Context** : Context the path distance is measured from (defaults to the querier).
- **AgentRadius** : Agent capsule radius used to plan the path that is measured (cm).

EnvQueryTest_LcmThreatExposure

Summed SweepPenalty along the item→threat segments. Float (lower = safer).

PROPERTIES

- **Context** : Threat context whose members the exposure is summed against (defaults to known threats).
- **NumSamplesPerSegment** : Number of sample points evaluated along each item->threat segment.

EnvQueryTest_LcmCoverScore3D

Fraction of context threats the item is occluded from. Float (higher = better cover).

PROPERTIES

- **Context** : Threat context the item must be occluded from to count as cover (defaults to known threats).
- **AgentRadius** : Agent capsule radius used for the occlusion LOS sweep to each threat (cm).

EnvQueryTest_LcmHeightAdvantage

Vertical advantage of the item over the context (high ground). Float.

PROPERTIES

- **Context** : Context the item's vertical advantage is measured against (defaults to known threats).

LcmFlightAnimInterface

Push contract for flight animation: the flight movement component broadcasts the archetype-agnostic flight state to any AnimInstance or owner that implements this interface, once per tick.

FUNCTIONS

- **ReceiveFlightState** : Pushed by ULcmFlightMovementComponent each tick with the current flight state. Implement in an AnimBP (or the pawn/mesh) to drive animation.

LcmFlightDynamics

Stateless flight/swim locomotion math: kinematic parity step plus the 6-DOF rigid-body cascade (generic / multirotor / bird / fish archetypes). Shared by the pawn flight movement component and the Mass steer processors so both run byte-identical physics.

LcmFlightMovementComponent

Opt-in pawn movement component that turns nav guidance velocities (RequestVelocity from FlyTo tasks) into realistic 6-DOF flight/swim motion via ULcmFlightDynamics, sweeps the transform, and broadcasts the anim state. Inert until bEnabled is set.

FUNCTIONS

- **RequestVelocity** : Feed the guidance command for this frame (world-space desired velocity). Persists until replaced or cleared, so a slower controller tick still produces continuous motion.
- **GetRigidState** : The full 6-DOF integrator state from the last step.
- **GetFlightState** : The animation / secondary-motion drive signals from the last step.
- **ClearGuidance** : Stop consuming guidance and coast (dynamics still decelerate the body).

PROPERTIES

- **bEnabled** : Master opt-in. Default false → the component is inert (base behaviour), guaranteeing byte-identical motion to before it was added.
- **FlightConfig** : Body/aero/control tuning. The LocomotionModel + Archetype selectors live at the top of this struct and drive which parameters are shown (only the selected model/archetype's fields appear).
- **bBroadcastAnimState** : [P6] Each tick, push the flight state to the pawn's AnimInstance (and the owner) if it implements ILcmFlightAnimInterface. Default on; costs nothing unless an implementer exists.
- **RigidState** : Persistent integrator state, seeded from the pawn at BeginPlay and re-synced from the updated component each tick (so external teleports and swept-collision corrections are respected).
- **FlightState** : Last-step anim outputs.
- **PendingDesiredVelocity** : Latest guidance command and whether one is active.
- **bHasGuidance** : True when RequestVelocity was called since the last tick. Gates the dynamics step so an unfed component coasts instead of integrating stale guidance.

LcmFlyingPawn

Note: APawn already inherits from INavAgentInterface, so re-listing it here triggers C4584 (duplicate base class). We just override the virtuals.

PROPERTIES

- **CollisionSphere** : Root sphere collision component for the flying pawn.
- **MovementComponent** : LCM Nav3D-aware floating movement component driving the pawn along nav paths.

LcmGameplayAbility_RequestPath

GameplayAbility wrapper around the LCM Nav3D async path request, gating it through the standard GAS cost/cooldown/cancellation pipeline (budget via ULcmNavAttributeSet) and broadcasting the resulting path on completion.

FUNCTIONS

- **OnApexPathFound** : Async FindPathAsync callback (game thread): populates the result, broadcasts OnPathReady, and ends the ability.

PROPERTIES

- **StartLocation** : Path-search start.
- **GoalLocation** : Path-search goal.
- **QueryOptions** : Forwarded to FindPathAsync verbatim.

- **BudgetCost** : Cost in PathRequestBudget consumed per activation. Default 1. Reduce for "cheap" replans (e.g. drift-triggered) and raise for expensive cross-chunk queries.
- **OnPathReady** : Fired on completion (success or failure). Subscribers should receive PathPoints by-const-ref.

LcmMassPathProxy

Per-request callback carrier (FindPathAsync's dynamic delegate can't hold the handle).

FUNCTIONS

- **OnPathFound** : FindPathAsync completion delegate: forwards the result (tagged with this proxy's entity + request id) onto the bridge's MPSC queue, then recycles.

LcmMassApexBridgeSubsystem

World subsystem bridging Mass entities to LCM Nav3D async CPU pathfinding: pools request proxies, queues completed results, and stores per-entity paths.

PROPERTIES

- **ProxyPool** : All allocated path proxies (owned/kept alive; FreeProxies holds the idle ones).

LcmMassCrowdSpawner

World-Partition-safe Mass spawner: never streams out, so its crowd's lifetime is not tied to the player's distance from the spawn point (fixes the "all agents disappear when the player leaves the area" WP trap).

PROPERTIES

- **SpawnBatchPerFrame** : [startup smoothing, opt-in] Spawn at most N entities per frame instead of the whole crowd in one frame. 0 (default) = stock engine behaviour. A large one-frame spawn (e.g. 6000 agents) hitches startup ~70 ms; spreading it (e.g. 500) smooths that frame-pacing spike. Only applies when "Auto Spawn On Begin Play" is enabled. NOTE: this does NOT fix the editor's "Handled ensure: CurrentPhase=..." Mass phase-manager spam; that is an engine-level UE 5.2 startup race (fires with 2 agents, on finite maps, independent of spawn load; handled + compiled out of Shipping), not a spawn-frame cost.

LcmMassFlowFieldSubsystem

Generates + caches one CPU SVO flow field per (agent chunk, quantised goal).

LcmMassFlowSteerTrait

[DEPRECATED: M3, 2026-07-03] Legacy flow-field crowd trait. Superseded by the unified **ULcmMassNavAgentTrait** ("LCM Nav3D Agent") with NavMode=FlowField, which composes a byte-identical archetype. Kept fully functional; prefer the unified trait for new content. Authoring trait for a flow-field-steered crowd agent.

PROPERTIES

- **GoalMode** : FixedLocation (use Goal) or TaggedActor (chase an actor by tag; the crowd follows it as it moves and the shared field re-solves on the move).
- **Goal** : Shared crowd goal (world space, FixedLocation mode). All agents with this goal share one field.
- **TargetActorTag** : Tag of the moving target actor (TaggedActor mode).
- **bChaseLead** : Lead the moving target by its velocity (intercept ahead of it).
- **LeadSecondsCap** : Upper bound on lead time (s).
- **MaxSpeed** : Movement speed (cm/s).
- **AgentRadius** : Collision radius (cm). Also the minimum agent size for field generation.
- **AvoidanceRadius** : Neighbours within this distance contribute local avoidance steering (cm).
- **AvoidanceStrength** : Weight of avoidance vs flow-following (0 = off, 1 = balanced).
- **MaxAcceleration** : Max acceleration (cm/s²). Smooth ease in/out; 0 = instant (legacy floaty). Applies only to PhysicsWeighted flight (LinearDirect is instant).

- **MaxTurnRateDeg** : Max turn rate (deg/s). The key knob: smooth banking turns vs robotic snapping. Applies only to PhysicsWeighted flight (LinearDirect is instant).
- **ArriveRadius** : Arrival / slow-down radius around the goal (cm).
- **FlightStyle** : LinearDirect (instant) vs PhysicsWeighted (turn-rate + acceleration inertia).
- **AvoidanceStyle** : None / Reactive (boids) / Predictive (ORCA, smooth mutual passing) / CBS.
- **AvoidanceTimeHorizon** : ORCA time horizon (s). Predictive avoidance only.
- **LaneBias** : Lateral lane-spread strength (0 = off). `r.LcmNav.Mass.FlowLaneBias` (>=0) overrides.
- **LaneDensityNorm** : Neighbour count at which lane spreading saturates.
- **bAnticipatory** : ANTICIPATORY navigation: route the crowd around where moving obstacles are HEADING (their predicted swept path), not just where they are now, so it goes OVER a side gap that is about to close instead of squeezing into it. Costs a little more (predicted-occupancy stamping + slightly more re-solving) and can detour a touch wider. Off = today's reactive field. (`r.LcmNav.Mass.Anticipatory` overrides this: 0 forces off, >=1 forces on.)
- **AnticipatoryStrength** : How wide a berth to give predicted obstacle occupancy (higher = earlier / wider detour).

LcmMassFlowSteerProcessor

Per-frame processor: samples each agent's shared flow field, blends in local avoidance / lane spreading / natural movement, and moves the agent.

LcmMassGoalTrait

Authoring trait: assign a fixed-location or tagged-actor goal to a Nav3D Agent so it auto-paths there (requires the "LCM Nav3D Agent" trait too).

PROPERTIES

- **GoalMode** : FixedLocation (use GoalLocation) or TaggedActor (path to an actor by tag).
- **GoalLocation** : World location goal (FixedLocation mode).
- **TargetActorTag** : Tag of the target actor (TaggedActor mode). First actor with this tag wins.
- **RepathToleranceCm** : Re-issue the path if the resolved goal drifts more than this (cm).
- **ArrivalToleranceCm** : Treat the agent as "arrived" within this distance; stop re-pathing (cm).
- **ReissueIntervalSeconds** : Minimum seconds between path re-issues per agent (anti-flood safety).
- **bEnableGoalPrediction** : [Chase] Lead a moving TaggedActor target by its velocity so agents intercept rather than trail. TaggedActor mode only. Master kill switch: `r.LcmNav.Mass.Chase.Enable`.
- **LeadSecondsCap** : [Chase] Upper bound on lead time (s). Override fleet-wide with `r.LcmNav.Mass.Chase.LeadSecondsCap` (>=0).

LcmMassGoalProcessor

PrePhysics processor: resolves each agent's goal (location or tagged actor) and writes it to the path-request fragment, flipping Status to Pending.

LcmMassNavISMRenderTrait

Authoring trait: drop on the Mass config, set a mesh, get instanced visuals.

PROPERTIES

- **Mesh** : Mesh drawn (instanced) at each agent's transform.
- **UniformScale** : Uniform scale applied to each instance.

LcmMassISMRenderSubsystem

Owens the host actor + per-mesh ISM components; accumulates/commits instances.

PROPERTIES

- **HostActor** : Hidden actor that hosts the ISM components (lazily spawned).
- **MeshToISM** : One ISM component per static mesh used by the crowd.

LcmMassISMRenderProcessor

FrameEnd processor that streams agent transforms into the render subsystem.

LcmMassNav3DTrait

[DEPRECATED: M3, 2026-07-03] Legacy path-based agent trait. Superseded by the unified [ULcmMassNavAgentTrait](#) ("LCM Nav3D Agent") with NavMode=PathFollowing, which composes a byte-identical archetype. Kept fully functional so existing configs (DA_MassAgent_Apex) keep working; prefer the unified trait for new content.

Authoring trait: attaches the path nav fragments and exposes radius/speed, pathfinding, and natural-movement settings.

PROPERTIES

- AgentRadius** : Collision radius forwarded to the GPU solver (cm).
- MaxSpeed** : Maximum movement speed for the steering processor (cm/s).
- MobilityClassIndex** : Mobility-class index (reserved for per-class cost rules; 0 = default).
- AvoidanceRadius** : Neighbours within this distance contribute local avoidance steering (cm).
- AvoidanceStrength** : Weight of avoidance vs path-following (0 = off, 1 = balanced).
- bUseGPUPathfinding** : CPU async (default, stable, scales) vs GPU batched dispatch. The Mass equivalent of the BT FlyTo "Use GPU Pathfinding" toggle.
- GPUSolverMode** : [ModeUI] Which GPU solver runs this agent class's queries when bUseGPUPathfinding is true. Project Default follows the global r.LcmGPU.AStarMode CVar; explicit modes pin the dispatch (grid-fast Mode 0/1 vs any-angle Mode 2/3). Ignored for CPU queries.
- PathSolver** : Any-angle LazyThetaStar (smooth) vs grid-locked AStar (square turns).
- SmoothingMode** : Curved (CatmullRom) / shortcut (Linear) / raw staircase (None).
- MaxAcceleration** : Max acceleration (cm/s²). Limits speed change so agents ease into motion.
- MaxTurnRateDeg** : Max turn rate (deg/s). The key knob: replaces robotic square turns with smooth arcs. Lower = wider, lazier turns; 0 = instant snapping (legacy).
- LookAheadDistance** : Pure-pursuit look-ahead distance (cm). Larger = smoother cornering.
- ArriveRadius** : Arrival / slow-down radius around the goal (cm).
- FlightStyle** : LinearDirect (magic/biological, instant) vs PhysicsWeighted (inertia).
- AvoidanceStyle** : None / Reactive (boids) / Predictive (ORCA, smooth passing) / CBS.
- AvoidanceTimeHorizon** : ORCA time horizon (s). Used only by Predictive avoidance.
- LaneBias** : Density-aware lateral lane-spread strength (0 = off).
- LaneDensityNorm** : Neighbour count at which lane spreading saturates.
- bObstacleReactivity** : Re-plan when a moving obstacle invalidates the route this agent is following. Global kill switch: r.LcmNav.Mass.ObstacleReactivity.

LcmMassNavAgentTrait

[M3] Unified authoring trait: one trait, a NavMode selector. PathFollowing composes the path archetype (NavAgent + PathRequest + PathResult); FlowField composes the flow archetype (FlowField state + a const shared config fragment). Mode-specific properties hide via EditCondition on NavMode.

PROPERTIES

- NavMode** : Movement policy. PathFollowing = per-agent A* / GPU path; FlowField = shared crowd steering field. Drives which fragments + config + tag the trait adds.
- Compute** : [P4] CPU vs GPU compute; governs BOTH modes. CPU = deterministic default; GPU = batched dispatch (PathFollowing) / solve-offload (FlowField, P5); Auto = CPU unless the workload favours GPU (large SVO / many goals / big N) and an RHI is present. Replaces the old PathFollowing-only Use-GPU bool.
- AgentRadius** : Collision radius (cm).
- MaxSpeed** : Maximum movement speed (cm/s).
- AvoidanceRadius** : Neighbours within this distance contribute local avoidance steering (cm).
- AvoidanceStrength** : Weight of avoidance vs path/flow following (0 = off, 1 = balanced).
- MaxAcceleration** : Max acceleration (cm/s²). 0 = instant.

- **MaxTurnRateDeg** : Max turn rate (deg/s). Replaces robotic square turns with smooth arcs. 0 = instant.
- **ArriveRadius** : Arrival / slow-down radius around the goal (cm). NOTE: the legacy traits defaulted differently per mode (path 80 / flow 150); this unified default is 100, so set it explicitly if migrating a config that relied on the old default.
- **FlightStyle** : LinearDirect (instant, no inertia) vs PhysicsWeighted (turn-rate + accel inertia).
- **AvoidanceStyle** : None / Reactive (boids) / Predictive (ORCA) / CBS. NOTE: the legacy Flow Steer trait defaulted to Predictive (ORCA); this unified default is Reactive (matching the legacy path trait), so pick Predictive explicitly for smooth mutual passing in dense flow crowds.
- **AvoidanceTimeHorizon** : ORCA look-ahead horizon (s). Predictive avoidance only.
- **LaneBias** : Density-aware lateral lane-spread strength (0 = off).
- **LaneDensityNorm** : Neighbour count at which lane spreading saturates.
- **MobilityClassIndex** : Mobility-class index (reserved for per-class cost rules; 0 = default).
- **GPUSolverMode** : GPU solver mode for this agent's queries when the GPU path is taken (Compute != CPU). ProjectDefault follows r.LcmGPU.AStarMode.
- **bUseGPUPathfinding** : [P4 legacy, prefer Compute] Old PathFollowing-only GPU toggle. Retained so pre-P4 DataAssets keep their setting: when Compute is Auto and this is true the agent resolves to GPU. Hidden unless Compute is Auto.
- **PathSolver** : Any-angle LazyThetaStar (smooth) vs grid-locked AStar (square turns).
- **SmoothingMode** : Curved (CatmullRom) / shortcut (Linear) / raw staircase (None).
- **LookAheadDistance** : Pure-pursuit look-ahead distance (cm). Larger = smoother cornering.
- **bObstacleReactivity** : Re-plan when a moving obstacle invalidates the route being followed.
- **FlightConfig** : Body / aero / control tuning for the 6-DOF model, the SAME model as the pawn Lcm Flight Movement Component. The LocomotionModel + Archetype selectors are at the top of this struct and drive which parameters show; default LocomotionModel = Kinematic → the crowd moves exactly as before until you opt in.
- **FlightDynamicsLODDistance** : [P8] Dynamics LOD distance (cm): beyond this from the viewer, agents use the cheap kinematic integrator (full 6-DOF only near camera). The scale knob for large crowds. 0 = always full. Kill switch: r.LcmNav.Mass.FlightLOD.
- **bAnticipatory** : Route around where moving obstacles are HEADING (predicted swept volume).
- **AnticipatoryStrength** : Detour strength around predicted occupancy (higher = wider berth).
- **bAvoidThreats** : Route the crowd AROUND actors tagged "ApexThreat" (danger zones). NOTE: agents DETOUR around threats, which can slow arrival - measured -30% threat crossings but 13-25% fewer arrivals on tight corridors. Enable for threat/danger gameplay.
- **ThreatDetourStrength** : Threat detour strength (higher = wider berth around danger zones).
- **bIncrementalField** : [Perf] D* Lite INCREMENTAL corridor field for large / churn-heavy crowds (re-cost only changed cells; safe full-rebuild fallback). Shared per goal.
- **bAsyncFlowSolve** : [Perf P2] Solve the flow field on a WORKER thread (removes solve spikes from the frame; the field publishes a frame later). Recommended for large crowds.

LcmMassNavDebugProcessor

Debug processor: draws a per-agent sphere (coloured by path status) and a line to its current waypoint. Gated off behind r.LcmNav.Mass.DebugDraw.

LcmMassPathRequestProcessor

PrePhysics. Walks entities whose FLcmMassPathRequestFragment is Pending and whose replan cooldown has elapsed, hands each to ULcmMassApexBridgeSubsystem, then flushes the bridge so chunk-saturated batches dispatch this tick.

LcmMassObstacleReactivityProcessor

PrePhysics: dynamic-obstacle reactivity. For each agent that is following a completed path (Status==Complete) with bObstacleReactivity, checks the manager's nav dirty-region ledger: if any hard-block region recorded since this path was

built intersects the agent's REMAINING route, flips Status→Pending so the request processor re-plans it (rate-limited by a per-agent cooldown). World-space box test → identical for finite and infinite worlds.

Ordering: ExecuteAfter(goal processor) so an authoritative goal-drift repath wins; ExecuteBefore(request processor) so the Pending flip dispatches the same PrePhysics pass. (Mass schedules within a phase by fragment-access conflicts, NOT registration order, so this explicit order is required: both this and the request processor write FLcmMassPathRequestFragment.)

LcmMassPathFollowSteeringProcessor

During-physics movement. Steers each entity with a valid path toward its current FLcmMassPathResultFragment::NextWaypoint at FLcmMassNavAgentFragment:: MaxSpeed by integrating the transform. (v1: see file header for the deferred flow-field-sampling upgrade.)

LcmMassPathResultProcessor

PostPhysics. Two jobs: (1) drains GPU path results that came back this frame and writes them into the entities' result fragments (storing the full path on the bridge); (2) advances CurrentWaypointIndex when an entity has reached its NextWaypoint, pulling the next waypoint from the bridge and transitioning the request to Idle when the path is exhausted.

LcmMassPerceptionProcessor

PrePhysics processor: runs the SVO-LOS perception core each LOD-throttled tick, resolving the closest visible hostile per sensor and writing its perceived-threats fragment.

LcmMassPerceptionTrait

Authoring trait: makes an LCM Nav3D-Mass agent both a sensor and a perceivable (faction-based SVO-LOS perception), exposing sight/FOV/eye-height settings.

PROPERTIES

- **FactionId** : Faction id (different factions are mutually hostile / perceivable).
- **SightRadius** : Maximum sight distance (cm).
- **FOVDegrees** : Full field-of-view angle (degrees).
- **EyeHeightOffset** : Eye height offset from the agent origin (cm).
- **AgentRadius** : Agent radius for the SVO LOS walk (cm).
- **PerceptionIntervalFrames** : Perceive once every N frames (LOD throttle).

LcmMassRepLODCollectorProcessor

Auto-registering LOD collector (fills FMassViewerInfoFragment) scoped to LCM Nav3D representation agents. Engine's UMassLODCollectorProcessor is auto-register OFF.

LcmMassRepVisualizationLODProcessor

Auto-registering visualisation-LOD processor (computes LOD, clears the distance-cull tag) scoped to LCM Nav3D representation agents.

LcmMassRepVisualizationProcessor

Auto-registering visualisation processor (assigns StaticMeshInstance representation + registers ISM instances) scoped to LCM Nav3D representation agents. UMassUpdateISMProcessor (already auto-register ON) then pushes the transforms.

LcmMassRepresentationTrait

Preconfigured UMassVisualizationTrait: LOD-switched crowd rendering that draws the instanced static mesh at all visible LODs by default (no "see nothing" trap), and self-provides the LOD-collector + actor fragments so it works as one trait.

LcmMpTestObstacle

Replicated test obstacle for the MP validation scenario. Everything the CLIENT replica needs (movable mobility, blocking collision, the cube mesh) is set in the CONSTRUCTOR so replicas construct identically: runtime property writes on the server (mobility, mesh) do NOT replicate, which is exactly the pitfall the first validation run caught.

PROPERTIES

- **Mesh** : Replicated cube visual/collision mesh (constructor-configured so replicas construct identically).

LcmMpTestStaticWall

Replicated STATIC-mobility wall: P9.6 auto-attach REJECTS it (movable=0), so no host tracks it. The ONLY way peers learn its footprint blocks navigation is the **UpdateDynamicObstacleReplicated** multicast replay. That isolation is what makes it the multicast-transport probe.

PROPERTIES

- **Mesh** : Non-replicated wall mesh. Peers learn its nav footprint only via the obstacle multicast replay (the transport probe).

LcmMpValidationSubsystem

Headless multiplayer validation driver (active only under -LcmMpTest).

FUNCTIONS

- **OnSanityPath** : G1 sanity-path handler (server-local FindPathAsync).
- **OnReplicatedPath** : G2 replicated-request handler (fires via Client_DeliverPath on clients).
- **OnDigestPath** : G4 digest-chain handler (local CPU queries, sequential).

PROPERTIES

- **Manager** : The world's navigation manager, resolved once when the run activates.
- **Cube** : Server-spawned replicated moving obstacle (G2 probe).
- **Wall** : Server-spawned non-replicated wall (G3 multicast-transport probe).

LcmNavAreaModifier

Designer-placed 3D volumetric cost-overlay actor; applies a cost multiplier to LCM Nav3D pathfinding wherever the agent's leaf is inside its box (water, storm, danger, or preferred-lane volumes that stack multiplicatively).

FUNCTIONS

- **GetCostMultiplier** : Returns this volume's cost multiplier applied to agents inside the box.
- **GetBounds** : World-space FBox covering the modifier volume.

PROPERTIES

- **BoundsBox** : Box bounds (extents around the actor's location). Use the editor gizmo to size the volume.
- **CostMultiplier** : Cost multiplier applied to LCM Nav3D pathfinding when the agent is inside the box. 1.0 = no effect; 2.0 = costs twice as much; 0.5 = preferred lane. Default 2.0.
- **bAutoRegisterWithApex** : Auto-register with the LCM Nav3D extension registry on BeginPlay.
- **AreaTag** : Optional gameplay tag for designer categorisation (e.g. "Water", "Storm", "Danger"; not used by solver in v1).

LcmNavAreaQueryLibrary

Blueprint function library exposing the LCM Nav3D nav-extension registry so designers can query cost overlays and registered NavLinks from BP graphs.

FUNCTIONS

- **GetRegisteredLinkCount** : Count of currently-registered NavLinks.
- **GetRegisteredAreaCount** : Count of currently-registered NavArea modifiers.
- **GetCostMultiplierAt** : Cost multiplier at the given world position. Returns 1.0 when no modifier applies; multiplies when overlapping modifiers stack.
- **GetCostMultiplierAlongPath** : Returns the sum of per-leg cost multipliers along the waypoint chain, weighted by leg length. Useful for "is this path expensive" predicates without rerunning the solver.
- **GetAllLinkEndpoints** : All registered link endpoints as FVector pairs (Start, End). Useful for debug visualisation.
- **FindNearestNavLink** : Finds the nearest registered NavLink to **WorldPos** within **MaxDistanceCm**. Returns nullptr if none.

LcmNavArmTagLibrary

Blueprint-callable wrapper around the **r.LcmAblation.ArmTag** CVar so designers can stamp per-task arm tags on telemetry rows for in-game A/B analysis without using the console.

FUNCTIONS

- **SetArmTag** : Sets the current **r.LcmAblation.ArmTag** value. Empty FName clears it.
- **PushArmTag** : Atomically swap the arm tag to **NewArmTag**, returning the previous value. Caller pairs with **PopArmTag** to restore on task exit.
- **PopArmTag** : Restore a previously-pushed arm tag.
- **GetCurrentArmTag** : Reads the current **r.LcmAblation.ArmTag** value. Empty FName when unset.

LcmNavAttributeSet

GAS attribute set surfacing per-agent path-request budgets so projects can throttle expensive LCM Nav3D queries through the standard GameplayEffect pipeline.

FUNCTIONS

- **OnRep_PathRequestBudget** : RepNotify for PathRequestBudget; mirrors the replicated change to GAS.
- **OnRep_MaxPathRequestBudget** : RepNotify for MaxPathRequestBudget; mirrors the replicated change to GAS.

PROPERTIES

- **PathRequestBudget** : Current path-request budget. Consumed by 1 per successful ULcmGameplayAbility_RequestPath activation. Default 5.
- **MaxPathRequestBudget** : Hard cap clamp for regen. Default 5.

LcmNavBridgeSubsystem

World subsystem that auto-registers an ALcmNavigationDataProxy with UNavigationSystemV1 once both the LCM Nav3D manager and nav system are live, so stock UE5 nav queries route through LCM Nav3D.

PROPERTIES

- **Proxy** : Spawned proxy actor registered with the nav system; destroyed on Deinitialize.

LcmNavigationChunk

Self-contained chunk actor: voxelizes its own region in-place and serves intra-chunk pathfinding queries directly. Pre- / pre-WorldPartition variant of the chunk pipeline; superseded in infinite-world flows by **ALcmSV0StreamingProxy** (which carries a pre-baked SVO data asset and registers with **ALcmNavigationManagerSV0::ActiveVirtualChunks**). Retained for editor-time bake workflows and tutorial scenes.

FUNCTIONS

- `DrawDebugChunk` : Helper to visualize this specific chunk
- `BuildChunk` : Builds the SVO *only* inside `ChunkBounds`

PROPERTIES

- `bIsGPUGenerated` : Remembers if this chunk was built using the Compute Shader
- `ChunkBounds` : Defines the area this chunk covers (Center and Extent)
- `RenderComponent` : The Native Render Thread Visualizer
- `SerializedSVOData` : On-disk-persistable byte stream of the baked SVO data. Populated by `BakeDataToMemory` ; consumed by streaming-proxy load paths.
- `BakedDataSize` : Human-readable size of `SerializedSVOData` (e.g. "342 KB"). Editor- facing diagnostic.
- `CachedMinVoxelSize` : --- MEMORY VARIABLES ---
- `CachedClearancePadding` : Clearance padding (cm) used at last build. Persisted so a rebake reuses the same value unless explicitly changed by the caller.
- `SavedObstacleChannels` : Collision channels treated as obstacles at last build. Persisted so a rebake reuses the same channel set unless overridden.

LcmNavigationDataProxy

`ANavigationData` subclass that exposes the LCM Nav3D SVO as a registered nav-data provider, routing stock `UNavigationSystemV1` path/project/raycast queries through the LCM Nav3D pathfinding solver.

LcmNavigationInvokerComponent

Attach this to the Player or VIPs. The SVO Manager will only generate voxel data for chunks within this radius.

PROPERTIES

- `GenerationRadius` : How far around this actor should we build navigation data? E.g., 10000 = Build chunks within a 100m radius.

LcmNavigationManagerSVO

LCMNav3D world singleton: the central navigation manager actor. Spawns the four subsystem components (`ULcmSVOSTreamingComponent` , `ULcmSVOGenerationComponent` , `ULcmPathfindingComponent` , `ULcmSVORenderComponent`); owns the SVO data containers (`FiniteData` for finite worlds, `ActiveVirtualChunks` for infinite worlds); owns `FLcmMacroGraph` + `FLcmPortalBuilder` ; hosts the multicast RPCs for dynamic-obstacle invalidation and flow-field-ready notifications. Exactly one instance per `UWorld`; auto-discovered via `UGameplayStatics:: GetActorOfClass` .

FUNCTIONS

- `UpdateDynamicObstacleReplicated` : [Network replication contract, 2026-05-20] -----
----- Server-authoritative entry: applies the dynamic-obstacle update locally AND multicasts it to all connected peers (clients + listen-server). Safe to call from any context: it becomes a no-op + warning when called on a client (clients route writes through the server). The flow-field-ready signal is a multicast-only callback path. The GPU flow-field buffers are NOT replicated: this RPC informs peers that a goal at (GoalLocation, ChunkKey) is reachable on the server, so they can prewarm their own local generation or trigger gameplay.
- `UpdateDynamicObstacle` : Call this when a large object moves to update the nav graph locally.
- `ToggleDrawDebugVolumes` : Flips `bDrawDebugVolumes` via `SetDrawDebugVolumes`. Convenience entry for a single-key toggle binding.
- `SetMacroTopologyCell` : [Procedural Topology] Called by your procedural-generation system (or World Partition minimap scanner) to mark a coarse cell blocked/unblocked.
- `SetDrawDebugVolumes` : Runtime show/hide for the SVO debug voxel volumes. Sets `bDrawDebugVolumes` and ALWAYS rebuilds the render proxy. The scene proxy captures the flag at build time (`FLcmSVOSceneProxy::bForceDraw`), so a rebuild is required for BOTH show and hide: toggling the bool alone does nothing. Bind to a key for in-game/PIE show/hide.

- **Multicast_UpdateDynamicObstacle** : Server-only entry: multicasts a dynamic-obstacle update to every connected client so each peer's **ALcmNavigationManagerSVO** replays **UpdateDynamicObstacle(Bounds)** locally (RCU clone is identical on every host). Clients that fire this directly are warn-and-ignored.
- **Multicast_FlowFieldReady** : Server-only entry: multicasts a "flow-field finished generating" notification to clients so they can prewarm a local sample for the given goal + chunk. The GPU buffers are NOT replicated; clients regenerate on demand using shared cache keys.
- **ForceRedrawSVO** : Keep the core function so Blueprints can still call it at runtime if needed
- **FindPath** : Calculates a path from Start to End using the SVO.
- **FindCover** : Find a hidden spot safe from the Enemy.
- **DrawDebugSVO** : Visualizes the octree in the viewport (Green = Free, Red = Blocked). One-shot manual call: flushes prior persistent lines and draws with a 10 s lifetime. The per-frame Tick path uses **DrawDebugSVOTick** instead.
- **DrawDebugNeighborsAt** : Visualizes neighbors of the node at the specific location (for debugging connectivity).
- **DrawDebugInfiniteWorld** : Renders the real-time state of the infinite world systems.
- **DebugDrawFlowField** : Flow Field Debug Visualization Generates a temporary Flow Field to the TargetLocation and draws it as a 3D Vector Heatmap. Green = Low Cost (Near Target), Red = High Cost (Far from Target).
- **BuildSVOInEditor** : Editor-time SVO build (button in the Details panel). Populates FiniteData from live collision geometry WITHOUT entering PIE, so the stock UE EQS Testing Pawn, and any LCM Nav3D EQS query built from the generators & tests, resolves the SVO and renders true-3D candidate points in the editor viewport, exactly the in-editor workflow Recast navmesh gives for 2.5D EQS. Finite mode only; warns + no-ops while playing (the SVO already bakes on BeginPlay) and in infinite-world mode (which streams chunks at runtime). Press once, then drop an **EQSTestingPawn**. Surfaced as the prominent "Build SVO" button in the editor module's banner (**FLcmNavManagerDetails**); **BlueprintCallable** so it can also be triggered from script. Intentionally NOT a **CallInEditor** button, which rendered a detached row below the property sections instead of in place.
- **BuildSVO** : Clears old data and builds a new SVO from scratch (Finite Mode only).

PROPERTIES

- **bIsInfiniteWorld** : World Type. OFF = a single fixed-bounds SVO built for the whole level (best for arenas / linear levels). ON = a streamed, unbounded world using Hierarchical HPA* (chunks generate on-demand around invokers). Drives the Generation Strategy below.
- **GenerationMode** : Auto-derived from the World Type, NOT set by hand. An infinite world always uses the Hierarchical HPA* strategy (macro routing + on-demand chunk streaming), the only infinite mode; a finite world is always Static Finite. Shown read-only so it can never be set to a value that contradicts the World Type (e.g. infinite + Fixed Bounds, which would silently break streaming). Tick "Infinite World" above to switch strategy.
- **bAutoBuildOnPlay** : Should the SVO automatically build itself when the game starts? (Finite mode only)
- **WorldExtent** : The total size of the navigable world (Cube width). Only used for Finite worlds.
- **ChunkSize** : Size of each chunk in World Units (e.g., 50,000 for 500m chunks). Must match World Partition Grid.
- **MinVoxelSize** : The smallest possible voxel size. Recursion stops here. (e.g., 100 = 1 meter)
- **ClearancePadding** : The padding added to voxel collision checks. Set this to your largest agent's radius.
- **ObstacleQueryChannels** : Physics collision channels to treat as solid walls.
- **AllowedObstacleClasses** : If empty, ALL overlapping geometry is voxelized. If populated, ONLY these actor classes are voxelized.
- **CoarseCellSize** : The size of a single topological macro-cell. 2500 units (25 meters) is the AAA standard for coarse LOD pathfinding.
- **bUseFlowFields** : When true, the request classifier can elect the GPU flow-field path instead of per-agent A* for swarms of $\geq$ **FlowFieldThreshold** agents sharing a goal. Off by default; opt-in for swarm gameplay.
- **FiniteObstacleUpdateInterval** : [Perf fix, 2026-06-07] FINITE mode: minimum seconds between dynamic-obstacle SVO applications. In finite mode every **UpdateDynamicObstacle** deep-clones the ENTIRE world SVO (RCU copy-on-write) and forces a full-world GPU graph re-upload on the next query. With event-driven obstacle components flushing every ~33 ms per moving actor, that was a clone+upload storm ("massive fps drop while anything moves"). Updates arriving inside the window are UNIONED and applied as ONE clone+swap at the trailing edge, so the final state always lands. 0 = legacy immediate behaviour. Infinite mode is unaffected (per-chunk updates are cheap by construction).

- **FlowFieldThreshold** : Minimum number of agents sharing a goal before the classifier elects the GPU flow-field path instead of per-agent A* (requires bUseFlowFields).
- **FlowFieldClusterCellSize** : Voxel-quantization grid size (cm) used to bucket flow-field goal positions into cache keys. Two requests whose goals fall into the same cell share a cached flow field. Larger values → more sharing (cheaper) but coarser direction snapping.
- **bUseGPUVoxelization** : Uses Compute Shaders for lightning-fast voxelization. Disable only if targeting low-end mobile.
- **MaxGenerationsPerFrame** : How many chunks can we voxelize per frame? (1 is usually best to avoid Game Thread stutters).
- **MaxFlowSolvesPerFrame** : [Perf P3a] Cap goal-anchored flow-field solves STARTED per frame (spreads the GT cost so a burst cannot spike the frame). 0 = unlimited. `r.LcmNav.Mass.MaxCorridorSolvesPerFrame` overrides (>=0).
- **MaxPathRequestsPerFrame** : [Perf] Max PathFollowing path requests dispatched per frame (spreads a spawn burst across frames; requests are async). `r.LcmNav.Mass.MaxPathsPerTick` overrides (>=0).
- **MaxRequestsPerFrame** : How many paths can we calculate per frame? (50 is a safe number to keep FPS high).
- **bChunkQuantizedMacroCache** : Share one cross-chunk macro route across a crowd heading to the same goal, instead of recomputing the identical chunk-level route per agent. This is the big win for LARGE Mass/AI crowds in infinite/hierarchical worlds: it removes the per-agent macro-pathfinding cost that otherwise spikes the game thread when many agents spawn or replan at once (measured ~30x cheaper request processing, worst-frame 97ms → 12ms at 2000 agents). The macro route is chunk-level; each agent's exact start/end are still applied per agent, so paths stay correct. Leave OFF to reproduce legacy bit-exact pathfinding (e.g. for deterministic benchmarking). Runtime override: `r.LcmNav.MacroCache.ChunkQuantized` (force-on). Infinite/hierarchical only.
- **bDrawDebugVolumes** : If true, automatically renders the voxel grid when generation finishes.
- **bDrawInfiniteWorldDebug** : If true, draws Invoker Bubbles, the Generation Queue, and loaded chunks in real-time.
- **bDrawPathfindingDebug** : Master switch for Agent Flight Paths and Waypoints.
- **bForceRedrawSVO** : Check this box in the editor to force the component to rebuild the mesh. It will instantly uncheck itself.
- **DebugRedrawCoalesceSeconds** : Minimum time between debug-mesh redraws (coalesce cadence). Forwarded to the SVO debug renderer; higher = fewer rebuilds when many obstacles move at once.
- **MacroGraph** : A lightweight map of every chunk in the world (HPA*).
- **RenderComponent** : The Central Renderer used to draw the SVO via the GPU Render Thread.
- **DynamicMacroTopology** : [Procedural Topology] Thread-safe map of the global layout of massive blocking geometry (coarse-grid coordinate → blocked). Populated by `SetMacroTopologyCell`.
- **StreamingComponent** : SUBSYSTEM COMPONENTS
- **GenerationComponent** : SVO voxelization scheduler subsystem. Pulls pending chunk-generation requests off the manager's queue and dispatches CPU/GPU voxelization.
- **PathfindingComponent** : Pathfinding request queue + dispatcher subsystem. Routes async **FindPathAsync** calls to the CPU worker pool or the GPU batched dispatcher, per the request options + election heuristic.

LcmNavLinkComponent

3D vertical NavLink component (subclass of `UNavLinkCustomComponent`) with designer-set world-space `LinkStart/LinkEnd` for ladders, shafts, and teleporters that Recast's surface-projected links cannot represent.

FUNCTIONS

- **IsVerticalLink** : True iff this link spans more than **MinVerticalCm** along Z: the 3D-advantage case (ladders, jump shafts).
- **GetLinkSpanCm** : Convenience: `distance(LinkStart, LinkEnd)`. Useful for designers comparing the link's geometric length vs its travel cost.

PROPERTIES

- **LinkStart** : World-space start point. For ladders/teleporters this is the bottom (or originating end if not vertical).
- **LinkEnd** : World-space end point. For vertical ladders this is the top.
- **bBidirectional** : When true (default), the link can be traversed Start→End AND End→Start. Set false for one-way patterns (teleporter, drop-down, etc.).

- `bAutoRegisterWithApex` : Auto-register with the LCM Nav3D link registry on BeginPlay so v2 solver integration discovers the link without buyer code.
- `TraversalCostCm` : Traversal cost (cm-equivalent) added to the path through this link. Default 0 = no penalty (geometric distance counts as-is).

LcmNavMovementComponent

UFloatingPawnMovement subclass with LCM Nav3D-aware defaults so stock UE5 path-following (UBTTask_MoveTo, MoveToActor) drives the flying pawn, plus an optional clearance brake that stops the pawn entering blocked SVO leaves.

PROPERTIES

- `bApplyClearanceBrake` : When true, every Tick we sample SVO clearance at the pawn's current location; if the leaf is blocked, we brake to zero velocity instead of pushing further. Default false, as it costs one O(1) leaf lookup per tick per pawn.

LcmNavProfiler

Drop-in per-cost nav profiler: place in a level and press Play. Captures the Mass nav per-cost breakdown each frame and writes a per-second CSV to Saved/Profiling/. Zero-config; enabling flips the otherwise-inert scoped timers in the nav code.

PROPERTIES

- `SampleIntervalSeconds` : Seconds between CSV rows (averaged over the window).
- `bAlsoLog` : Also print each row to the log (LcmNavProfiler) as it is written.
- `SpikeLogThresholdMs` : [startup-stall diag] Log a per-frame breakdown for ANY single frame whose time exceeds this (ms). Catches individual hitch frames that per-second averaging hides. 0 disables. The unattributed remainder (frame minus the nav buckets) = cost OUTSIDE the nav plugin (spawn / streaming / render).

LcmNavProfileSubsystem

Engine subsystem that loads JSON CVar tuning profiles at runtime, applying them atomically for hot-swappable A/B experiments without rebuilds.

FUNCTIONS

- `LoadProfile` : Reads `JsonPath`, parses as a flat CVar object, applies each (key, value) pair to the live console manager. Returns the count of successfully-applied CVars; -1 on parse failure (no partial apply). Logs per-CVar audit entries at LogTemp Display level with the (old → new) transition for traceability.

LcmNavServerRequestProxy

Per-request proxy UObject used by the server to bind a UFUNCTION callback into the Manager's dynamic `FOnLcmPathFound` delegate. Carries the `RequestID` so the `OnPathComplete` handler can deliver the result back to the originating client via the owning component's `Client_DeliverPath` RPC. One instance per in-flight server-side request; lifetime is bounded by the parent component's `PendingServerProxies` map.

FUNCTIONS

- `OnPathComplete` : Dynamic-delegate callback fired when the server-side `FindPathAsync` for this request completes. Looks up the owning component and routes the result to the originating client via `Client_DeliverPath`, then notifies the component to clean up server-side bookkeeping.

PROPERTIES

- `RequestID` : Monotonic id assigned by the originating client. Echoed back in `Client_DeliverPath` so the client correlates the reply.
- `OwningComponent` : Weak back-reference to the `ULcmNavReplicationComponent` that owns this proxy. Used to dispatch `Client_DeliverPath` on completion.

LcmNavReplicationComponent

Network replication contract component. Attaches to a `APlayerController` and owns the client↔server path-request RPC chain. Server is authoritative for every pathfinding mutation; client requests are routed via `Server_RequestPath`, computed via the manager's `FindPathAsync`, and replayed on the originating peer via `Client_DeliverPath` which fires the caller's original `FOnLcmPathFound` delegate.

FUNCTIONS

- `Server_RequestPath`: Server-side reliable RPC that runs `FindPathAsync` on the manager and arranges to route the result back to the originating client via `Client_DeliverPath`. Validated to reject malformed inputs from untrusted clients.
- `Client_DeliverPath`: Client-side reliable RPC that replays a server-computed path result on the originating peer. Looks up the stored delegate by `RequestID` and fires `OnPathFound(bSuccess, PathPoints)` exactly once. No-op if the client previously cancelled the request.

PROPERTIES

- `PendingClientRequests`: Client-side bookkeeping: id -> stored delegate. Cleared on delivery / cancel.
- `PendingServerProxies`: Server-side bookkeeping: id -> active proxy UObject, kept alive by UPROPERTY.
- `CachedManager`: Cached pointer to the world's `ALcmNavigationManagerSVO` singleton, resolved lazily by `ResolveManager` and reused for every RPC.

LcmNavVolume

Defines the playable area for offline SVO generation. Acts as the master bounds for slicing and baking `ALcmNavigationChunk` actors.

FUNCTIONS

- `BakeNavigationData`: Editor-only one-shot bake. Slices the volume bounds into the configured chunk grid, voxelizes each chunk on the game thread, and writes a `ULcmSV0DataAsset` per chunk into `OutputAssetPath`. Pairs with the runtime `ALcmSV0StreamingProxy` actor that loads each asset on level streaming.

PROPERTIES

- `ChunkSize`: Edge length (cm) of a single bake chunk. Must match the runtime manager's `ChunkSize` when `bSnapToGlobalGrid` is true.
- `MinVoxelSize`: Smallest voxel produced by SVO subdivision (cm). Tighter values produce finer paths but inflate the SVO node count cubically.
- `ClearancePadding`: Inflation (cm) applied to obstacle SAT collision checks at bake time. Set to the radius of your largest agent so the SVO carves out enough clearance for them.
- `bSnapToGlobalGrid`: If true, forces chunks to align to a global 3D grid (Required for World Partition). If false, spawns exactly ONE chunk that perfectly matches this volume's bounds (Best for Arena/Linear games).
- `BakeObstacleChannels`: Physics collision channels treated as solid obstacles at bake time. Empty means "all overlapping geometry".
- `BakeAllowedClasses`: Whitelist of actor classes to voxelize. If empty, all overlapping geometry on the configured channels is included.
- `OutputAssetPath`: Content path under which `ULcmSV0DataAsset` outputs are written by `BakeNavigationData`. Per-chunk assets get a `_X_Y_Z` suffix.

LcmPaperFigOverlayComponent

Persistent-mesh overlay primitive for paper-figure captures. Owned/spawned by the `LcmNavPaperFig` console commands; one instance per overlay layer ("Leaves", "Clearance"). Vertices are authored in WORLD space (identity primitive transform), matching `ULcmSV0RenderComponent`.

LcmPathfindingComponent

Pathfinding subsystem component owned by `ALcmNavigationManagerSV0`. Owns the per-frame async request queue, the CPU worker-task scheduler, and the GPU batched-dispatch driver. Routes a single Blueprint/C++ FindPath call to the right pipeline (CPU vs GPU Mode 0/1/2) based on the request options and the manager's election heuristic.

PROPERTIES

- `Manager`: Back-pointer to the owning navigation manager, resolved at runtime; transient (not serialized).

LcmPathFoundTrampoline

Generational dispatch trampoline for `FOnLcmPathFound`.

[Research Note: Generational Replan Handle (N5)]

`FOnLcmPathFound` is a UFUNCTION-style dynamic delegate, so its target cannot capture per-request state by value. This makes "drop the stale callback after a re-request" impossible at the dispatch boundary using the delegate itself.

Plumbing a TFunction-based overload through the existing 18 invocation sites (CPU worker + GPU path solver + GPU flow-field + early-failure branches) was rejected on risk + scope grounds. This trampoline achieves identical capture semantics in ~40 LoC, without modifying any of the existing pathfinding plumbing.

Usage:

- The consumer (e.g. `ULcmBTTask_FlyTo`) creates one trampoline per issued request via `NewObject<ULcmPathFoundTrampoline>(Outer)`.
- It anchors the trampoline in a UPROPERTY-rooted container on itself (`UPROPERTY TArray<TObjectPtr<ULcmPathFoundTrampoline>>`) so GC does not reclaim it while the async path request is in flight.
- It assigns a TFunction to `Forward` that captures `Generation` and any other per-request state by value, then binds the trampoline's UFUNCTION `Handle` to the dynamic `FOnLcmPathFound` delegate.
- When the worker invokes the delegate, `Handle` forwards to `Forward(...)`. The captured lambda performs the generation check and either dispatches to the real handler or drops the call.
- The lambda removes its trampoline from the anchor container after firing (or on `AbortTask`) so the UObject becomes eligible for GC.

Thread-safety: The worker marshals delegate invocation back to the game thread before firing, so all access to `Forward` and the anchor container is single-threaded.

FUNCTIONS

- `Handle`: UFUNCTION matching `FOnLcmPathFound`. Bound via `BindDynamic`.

LcmPathInterpolationComponent

Client-side component that smoothly interpolates a replicated AI pawn along its server-delivered waypoint path to hide RTT snap-jitter.

FUNCTIONS

- `ReceivePath`: Server delivers a fresh path; client begins interpolating along it. Pass an empty array to stop interpolation (e.g. on task end). Resets the internal arc-length parameter to 0, i.e. the client pawn snaps to `Waypoints[0]` on the next Tick. If you want continuity with the previous path, the caller should prepend the pawn's current position as the first waypoint.

PROPERTIES

- `MoveSpeed`: Cm/s the client interpolates the pawn along the path. Should match the server's FlyTo `FlightSpeed` for visual coherence.
- `PredictionMs`: Forward-prediction window (ms). The client extrapolates beyond its currently-known position by this much before lerping, which hides RTT-induced lag. Default 50 ms (typical LAN RTT).
- `bSkipOnAuthority`: When true (default), the component disables itself if the owning actor has Authority. Set to false for single-player smoke tests where you want client-side smoothing without a network role.

LcmSampleSmartObjectBase

Shared base

Owns the SmartObjectComponent + the runtime-built definition. Subclasses override `BuildDefinition` to install the correct behavior def into the single slot.

PROPERTIES

- `SmartObjectComponent` : SmartObjectComponent the runtime + StateTree task interact with.
- `Definition` : Per-actor runtime definition. Built in PostInitializeComponents via `BuildDefinition`.

LcmSampleSmartObject_Perch

Perch sample

PROPERTIES

- `DwellSeconds` : Mirrors the behavior-def DwellSeconds; copied at construction time.
- `RequiredClearanceCm` : Agent collision radius (cm) the perch must clear; mirrors the behavior def.

LcmSampleSmartObject_PatrolWaypoint

PatrolWaypoint sample

PROPERTIES

- `DwellSeconds` : Seconds the pawn dwells at this waypoint before moving to the next.
- `bChainCycles` : When true, the patrol loops back to the chain head after the last waypoint.

LcmSampleSmartObject_CoverPocket

CoverPocket sample

PROPERTIES

- `MaxAcceptableSweepPenalty` : Highest SweepPenalty (0..255) the cover slot may carry to count as safe.
- `FallbackSearchRadiusCm` : Radius (cm) within which FindBestCover searches when the placed pocket fails.

LcmSmartObjectBehaviorDef_CoverPocket

SmartObject behaviour definition for a designer-placed cover pocket, validated at claim time by SweepPenalty + threat-hint LOS with a FindBestCover fallback when the placed pocket has gone hot.

PROPERTIES

- `DwellSeconds` : Time the pawn stays in cover before considering the state complete. 0 = stay forever (use a parent transition condition to exit).
- `MaxAcceptableSweepPenalty` : Highest SweepPenalty (0..255) the slot leaf may carry to count as safe. Above this the StateTree task triggers the FindBestCover fallback. Default 32 = "lightly swept, mostly safe".
- `bUseThreatHint` : Direction hint (world-space) to validate cover against. The pocket is only valid if LCM Nav3D LOS from this point to the slot is blocked AND SweepPenalty is below the cap.
- `ThreatHintLocation` : World-space point the cover is validated against; the pocket is valid only if LCM Nav3D LOS from here to the slot is blocked.
- `FallbackSearchRadiusCm` : When the placed pocket fails validation, the task searches within this radius for a procedural cover via FindBestCover. 0 disables fallback (failure makes the task return Failed).
- `AgentRadius` : Agent radius for the LOS walker's clearance gate.

LcmSmartObjectBehaviorDef_PatrolWaypoint

SmartObject behaviour definition for one patrol waypoint; designers chain placed waypoints via NextWaypointTag to author a patrol route as content.

PROPERTIES

- DwellSeconds** : Dwell time at this waypoint before moving to the next.
- NextWaypointTag** : GameplayTag of the next smart object in the chain. Empty = end-of- chain; the parent state machine drops out of patrol on Empty.
- bChainCycles** : When true, the patrol restarts at the chain's head when the chain ends. When false, the pawn lingers at the final waypoint.
- bUseFacingHint** : Optional facing yaw to assume at this waypoint (degrees). When unset (kept default), keeps the inbound flight rotation.
- FacingYawDeg** : Yaw (degrees, world-space) the pawn faces at this waypoint when bUseFacingHint is set.

LcmSmartObjectBehaviorDef_Perch

SmartObject behaviour definition for a volumetric perch: fly here, hover or stand, and dwell. A 3D-only smart-object pattern LCM Nav3D's free-space SVO supports where surface-projected Recast navmesh cannot.

PROPERTIES

- DwellSeconds** : How long the pawn dwells at the perch after arrival.
- RequiredClearanceCm** : Agent collision radius the perch must accommodate. Used at claim time by **FLcmSTTask_UseSmartObject** to reject perches whose voxel clearance is below this threshold. Pass-through to the JFA clearance gate (same byte the Lazy Theta* LOS walker consumes).
- bUseFacingHint** : When true, the pawn rotates to face **FacingHint** after arrival. When false, the pawn keeps its inbound flight rotation.
- FacingYawDeg** : Yaw the perched pawn should face (degrees, world-space). Bound only when **bUseFacingHint** is true.
- FacingInterpSpeed** : Per-tick speed at which the pawn aligns to FacingYawDeg during dwell.

LcmSteeringMath

A stateless utility library that handles the heavy vector mathematics for agent flocking, avoidance, and path adherence.

LcmSTEvadeCBSRuntime

Per-task runtime UObject holding the CBS steering context for the Evade CBS StateTree task (kept off the value-type instance data for UObject lifetime).

LcmSTFlyToRuntime

Runtime proxy object that owns the async path state for one active StateTree FlyTo task instance. Created on EnterState and pinned by a hard **TObjectPtr** in instance-data (no AddToRoot; the StateTree's GC- tracked instance-data is what keeps it alive through the async dispatch window). Released on ExitState; **BeginDestroy** is a PIE-teardown safety net. Receives **OnPathFound** callbacks via dynamic delegate. Tick reads **bPathReady** / **bPathFailed** flags + the path array; the task USTRUCT remains stateless.

FUNCTIONS

- OnPathFound** : Dynamic callback bound on FindPathAsync dispatch. Writes to CurrentPath / CurrentPathIndex and flips bPathReady / bPathFailed. Honours PendingSpliceFromIndex / bIsWaitingForNextChunk for path-stitching.

LcmSTFollowFlowFieldRuntime

Per-task runtime UObject caching the computed waypoint chain and progress index for the Follow Flow Field StateTree task.

LcmSTUseSmartObjectRuntime

Runtime proxy for one active UseSmartObject task instance. Holds the claim handle, the per-tick phase, the resolved slot location, and the dwell timer. GC-pinned via a hard TObjectPtr in instance-data; no AddToRoot, per the crash-fix discipline.

PROPERTIES

- **ResolvedBehavior** : Behaviour definition resolved from **Subsystem.Use<BehaviorClass>** after arrival. Used to drive dwell + facing.

LcmSVODataAsset

A lightweight, streamable asset that holds pre-calculated SVO navigation data. Designed to be loaded asynchronously by World Partition.

PROPERTIES

- **ChunkCoordinate** : The mathematical grid coordinate this chunk belongs to
- **ChunkBounds** : The physical world bounds of this chunk
- **SerializedSVOData** : The raw, compressed binary data of the SVO tree
- **SerializedMacroNode** : [Persistent macro graph, 2026-05-18] -----
----- Optional macro-graph slice for this chunk: the just-finalized FLcmMacroNode (PortalEdges + InternalEdges + WorldCenter) captured after BuildTruePortals + PrecomputeIntraChunkEdges complete during the bake. Serialized via FLcmMacroNodeArchive (magic 'LCMN', V1). Empty array = legacy bake: the streaming proxy falls back to UpdateMacroNode + GeneratePortalsForChunk (fallback portals only; the runtime BuildTruePortals + intra-chunk-edge precompute still runs separately when neighbors load). Non-empty = the cold-build work is skipped and the precomputed macro node is injected directly into Manager->MacroGraph.Nodes[ChunkCoordinate]. Independent from SerializedSVOData: a backward-compatible addition; older baked assets deserialize with this field empty.

LcmSVOGenerationComponent

SVO voxelization scheduler component owned by **ALcmNavigationManagerSVO**. Tick-driven: pulls pending chunk-generation requests off the manager's queue, extracts geometry on the game thread, dispatches CPU or GPU voxelization on a background task, and publishes the finished SVO via RCU. Honors the far-chunk-LOD quality classification (full vs coarse).

PROPERTIES

- **Manager** : Back-pointer to the owning navigation manager, resolved at runtime; transient (not serialized).

LcmSVORenderComponent

Debug-visualization primitive component owned by **ALcmNavigationManagerSVO**. Draws the SVO leaves, portals, macro graph, and (optionally) flow-field arrows as a single procedural mesh. Toggled via the manager's **bDrawDebugVolumes** / **bDrawInfiniteWorldDebug** flags and the **r.LcmNav.RenderMode** CVar. Strictly opt-in: zero cost when disabled.

PROPERTIES

- **RedrawCoalesceSeconds** : Min seconds between scene-proxy rebuilds (debug visual cadence). Driven by the nav manager's "Debug Redraw Coalesce Seconds" (5. Debug); this internal subobject doesn't surface its own copy in the Details panel.

LcmSVOStreamingComponent

Chunk-streaming subsystem component owned by **ALcmNavigationManagerSVO**. Watches registered invokers, computes the union of full-bubble + promotion-edge AABBs each tick (far-chunk LOD), promotes / demotes chunks accordingly, and runs the LRU sweep that bounds total resident chunks at **r.LcmNav.MaxLoadedChunks**. Drives loading + eviction but does NOT touch SVO data directly: it dispatches generation work to **ULcmSVOGenerationComponent** and tears down via **ALcmSVOStreamingProxy**.

PROPERTIES

- **Manager** : Back-pointer to the owning navigation manager, resolved at runtime; transient (not serialized).
- **ActiveInvokers** : Set of invoker components currently driving streaming. Each invoker contributes a bubble of high-res chunks; the union determines which chunks load/unload. Mutated by **RegisterInvoker** / **UnregisterInvoker** . Transient: repopulated each PIE/runtime session.

LcmSVOSTreamingProxy

A lightweight proxy actor designed to be streamed by World Partition. Upon loading, it deserializes its SVO Data Asset and feeds it to the Navigation Manager.

PROPERTIES

- **BakedData** : The pre-baked data asset this proxy is responsible for loading

LcmTacticalQueryLibrary

Static Blueprint-callable wrappers for LCMNav3D tactical primitives.

All functions are pure CPU and synchronous: they execute on the calling thread (Blueprint VM thread, BT service tick, EQS test evaluation, etc.) and return immediately. None of them dispatch GPU work or async jobs. Designed to be called at EQS-test frequency (tens to hundreds per query per frame) without budget concerns.

FUNCTIONS

- **IsSegmentLOSClear** : Tests whether a straight line from Start to End is clear of unblocked SVO leaves for an agent of the given radius. Wraps the same Lazy Theta* neighbour-graph LOS walker used by Mode 2 pathfinding. Returns false if the SVO at Start cannot be resolved (out of bounds or unloaded chunk).
- **IsReachableSync** : Cheap reachability test. Returns true iff a path from Start to Goal exists. Implemented by calling the underlying A* with early-out; faster than FindApexPathSync when you don't need the waypoint list.
- **IsPositionBlocked** : Returns true iff WorldPos resolves to a blocked SVO leaf (or to no leaf at all: out-of-bounds and unloaded chunks both report blocked by default for tactical-query safety). Pair with GetSweepPenaltyAt when you need to distinguish "blocked" from "merely risky".
- **GetSweepPenaltyAt** : Reads the SweepPenalty byte at WorldPos. Higher values indicate the voxel was recently swept by a dynamic obstacle's predicted hull: a heuristic measure of "how dangerous is this position right now?". Returns 0 when the position resolves to a chunk that hasn't been loaded, when the voxel itself is blocked (treated as max danger by default in path costing; the caller may want to treat blocked separately via Is Position Blocked), or when no dynamic obstacle has touched it. Value range: 0 (clear).. 255 (maximum recent threat).
- **FindBestCover** : Wraps LcmSVOPathSolver::FindBestCover. Finds the best cover position for AgentLocation against ThreatLocation within SearchRadius (cm), preferring positions with line-of-sight broken to the threat. @param OutCoverLocation receives the best cover position when the function returns true. When the function returns false, this is left at ZeroVector.
- **FindApexPathSync** : Synchronous "is there a path from Start to Goal" probe. Wraps the same A* core that FindPathAsync uses but runs on the calling thread for use inside EQS tests that need to score candidates by LCM Nav3D-path distance (NOT euclidean) before the agent commits. Returns true when a path was found; OutPath receives the waypoint sequence. Returns false when start or goal don't resolve to a loaded chunk, when their containing voxels are blocked, or when no path exists between them. Pay-as-you-call cost: typical short-range paths complete in <0.1 ms, long-range cross-chunk paths in 1-5 ms. If you only need the boolean "is reachable?" answer without the waypoint sequence, prefer IsReachableSync (cheaper; it bails as soon as goal is popped).

Chapter 17 of the LCM Nav3D User Manual.

The complete manual is `LCM_Nav3D_Manual.pdf`, in this folder alongside every other document in the set. Start from `README.pdf`.

LCM Nav3D: True Volumetric 3D Navigation for Unreal Engine. Version 2.05 · Unreal Engine 5.8.

Copyright © 2026 L.Charitha Madhushanka. All Rights Reserved. Proprietary and confidential. Licensed, not sold; use is governed by the licence that accompanies this software. Unreal Engine is a trademark of Epic Games, Inc. All trademarks are the property of their respective owners.